Chapter 2 – Servlets-Part -2 - Overview of Java Web Projects and the web.xml

Contents

2.4.1 Steps to write Java Servlet with HTML using Eclipse IDE....2

2.4.2 Servlet Context....10

2.4.3 Servlet Config....12

2.4.4 ServletRequest Object and its Methods....16

2.4.5 ServletResponse Object and its Methods....20

2.4.6 Servlet and Filter configuration in web.xml....26

2.4.6.1 Configuring Servlets....26

2.4.6.2 Configuring Filters....30

2.4.7 Initialization and Context Parameters in web.xml....31

2.4.7.1 Initialization Parameters....31

2.4.7.2 Context Parameters....32

2.4.8 Project Directory Structure....33

2.4.9 Summary....35

Chapter 2 - Servlets - Overview of Java Web Projects and the web.xml

2.1 Objectives

The objectives of the course are the below

- To understand Servlet programming and the Servlet API
- To understand how to configure servlets and filters in web.xml.
- To understand the init param and context param configurations in web.xml and their importance.
- To understand project directory structure and its importance.

2.2 Audience

This document is designed for Java EE Learners and programmers with a need to understand the basics of Java Web projects and about web.xml file. The trainees who have understood the basics of Java technology and understood the basics of Web Applications are the intended audience.

2.3 Prerequisites

Associates should have gone through all the concepts of the core skills phase, specifially the core java concepts. Web Application Basic knowledge is also a prerequisite.

2.4 Course Content

2.4.1 Steps to write Java Servlet with HTML using Eclipse IDE

Let us discuss one simple application using servlet and HTML, please refer previous topics if you still have any doubts regarding servlets flow.

The example is dealing the following scenario:

- Ask the user for a color in a HTML - in our example it will be "Home.html"
- Display "Hello TCS" in the chosen color using a servlet - in our example it will be "ControllerServlet.java"

1. Create a new project

In the menu bar, File / new / Dynamic Web Project.

Name your project which should be consistent with the example, name it "ServletExample".

By choosing "Dynamic web project", eclipse creates the default folders hierarchy.

The folders hierarchy is shown in the "Project Explorer" view at the left side of the eclipse window.

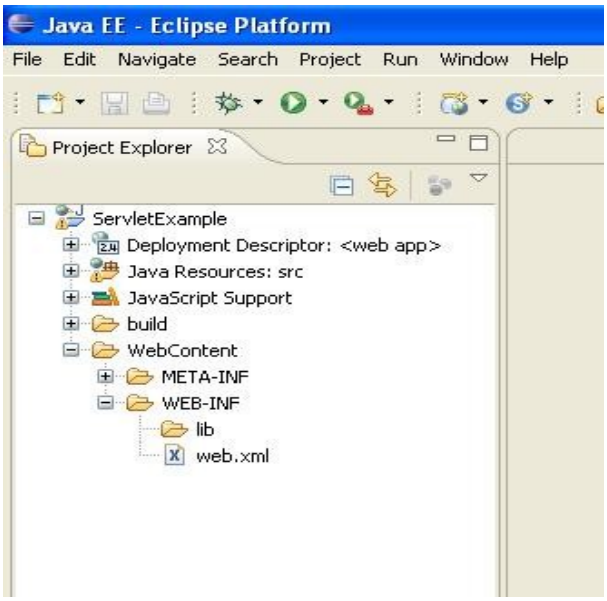


Figure 7: Project Explorer View (Eclipse)

If it is not shown, you can show it from the menu bar, Window / Show View / Project Explorer. It should look like the above figure. HTML, CSS files, images etc will be placed in the "WebContent" folder. Java files (Servlets) will be placed in the "Java Resources: src" folder.

2. Create the HTML file

In the "Project Explorer" view, Right-click "WebContent" / New / HTML.

Name your HTML - to be consistent with the example, name it "Home.html"

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
        pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>
        <head>
        <meta   http-equiv="Content-Type"   content="text/html;   charset=ISO-8859-1">
        <title> My first HTML   </title>
        </head>
        <body>
                <form action="ControllerServlet">
                        Please enter a color <br>
                        <input type="text" name="color"size="20px">
                        <input type="submit" value="submit">
                </form>
        </body>
</html>
```

3. Create the Servlet

In the "Project Explorer" view, Right Click "Java Resources: src" / New / Class

Name your class - to be consistent with the example, name it "ControllerServlet" and write the below code:

```
import java.io.IOException;

import javax.servlet.ServletException;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

import java.io.PrintWriter;



public class ControllerServlet extends HttpServlet {

  protected void doGet(HttpServletRequest request,

HttpServletResponse response) throws ServletException, IOException

  {

    // reading the user input

    String color= request.getParameter("color");

PrintWriter out = response.getWriter();

out.println (

                    "<!DOCTYPE html PUBLIC \"-//W3C//DTD HTML 4.01
Transitional//EN\"
```

```
        \"http://www.w3.org/TR/html4/loose.dtd\">\n" +
    "<html> \n" +
      "<head> \n" +
        "<meta http-equiv=\"Content-Type\" content=\"text/html;
          charset=ISO-8859-1\"> \n" +
        "<title> My first HTML  </title> \n" +
      "</head> \n" +
      "<body> \n" +
        "<font size=\"12px\" color=\"" + color + "\">" +
          "Hello TCS" +
        "</font> \n" +
      "</body> \n" +
    "</html>"
  );
 }
}
```

4. Define your servlet in "web.xml"

Open web.xml from the "Project Explorer" view, WebContent / Web-INF / Right Click web.xml / Open With / Text Editor. Define your web.xml code like below:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app id="WebApp_ID" version="2.4"
xmlns="http://java.sun.com/xml/ns/j2ee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
    http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
<servlet>
<servlet-name>ControllerServlet</servlet-name>
<servlet-class>ControllerServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>ControllerServlet</servlet-name>
<url-pattern>/ControllerServlet</url-pattern>
</servlet-mapping>
</web-app>
```

5. Add your project to Tomcat

- Right-click the Tomcat server / Add and Remove Projects; add your project
- Open the "Servers" view by clicking the "Servers" tab at the bottom of eclipse window. If the "Servers" tab is not shown, you can show it from the menu bar, Window / Show View/ Servers. Or Window / Show View / Other / Server / Servers.

35
17 In the "Servers" view, Right Click Tomcat-server record / add and Remove Projects.

6. Start "Tomcat":In the "Servers" view, Right Click Tomcat-server record / Start.

7. Test your project:

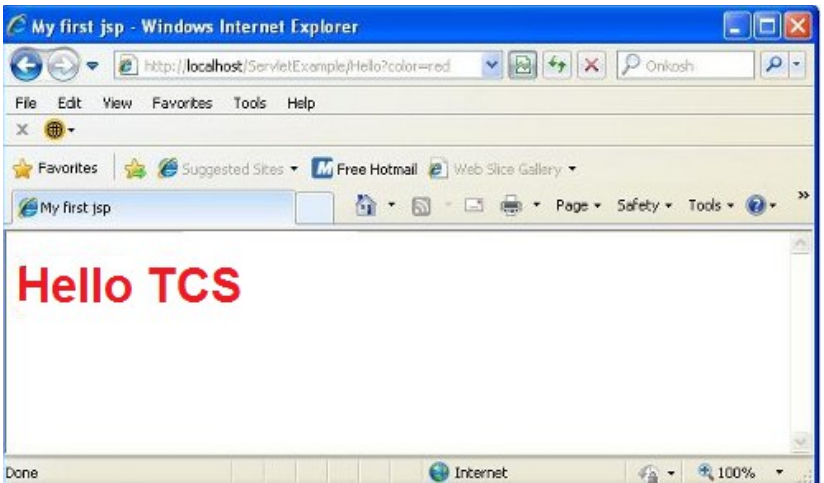
In your web browser (Internet Explorer or GoogleChrome) ,if the port is set to 8080 type http://localhost:8080/YourProjectName.In our example the URL is http://localhost:8080/ServletExample

Please note that Tomcat's default port is 8080(You can check your version's port from Tomcat-Installation-Directory / conf / server.xml)

Click your HTML file name shown in the directory listing - in our example it is Home.html and execute it.

The result should be

Figure 8: Sample Output



8. Set your project's welcome file in web.xml (let the server open "Home.html" once you open the project).

Open "web.xml" (WebContent / Web-INF / Right-click web.xml / Open With / Text Editor).

Set the "welcome-file-list" to be your home page - In our example it is Home.html. The code in "web.xml" should look like that:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app id="WebApp_ID" version="2.4"
xmlns="http://java.sun.com/xml/ns/j2ee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
    http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
<servlet>
<servlet-name>ControllerServlet</servlet-name>
<servlet-class>ControllerServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>ControllerServlet</servlet-name>
<url-pattern>/ControllerServlet</url-pattern>
</servlet-mapping></web-app>
```

Restart the server and test your project again using http://localhost:port/yourProjectName - in our example, the URL will be **http://localhost:8080/ServletExample/**

Note that the server does not recognize changes in "web.xml" except if you restart or start the server. If you have passed these steps successfully your Servlet would run successfully

2.4.2 Servlet Context

The ServletContext is an object that contains meta information about your web application. You can access it via the HttpRequest object, like this:

```
ServletContext context = request.getSession().getServletContext();
```

Context Attributes

Here is how we can store attributes in the servlet context as shown below

```
context.setAttribute("someValue", "aValue");
```

You can access the attributes again like this:

```
Object attribute = context.getAttribute("someValue");
```

The attributes stored in the ServletContext are available to all servlets in your application which means that the attributes are available to all visitors of the web application. The ServletContext attributes are still stored in the memory of the servlet container.

index.html

```
<form action="onContext" method="post">
    Example on ServletContext<br>
<input type="submit" value="Click Here">
</form>
```

OnServletContext.java

```
public class OnServletContext extends HttpServlet  {

protected void
    doPost(HttpServletRequestreq,HttpServletResponse res)

    throws ServletException,IOException

 {

    PrintWriter pw=res.getWriter();

    res.setContentType("text/html");

    ServletContext context=getServletContext();

       String s1=context.getInitParameter("number1");

       String s2=context.getInitParameter("number2");

pw.println("n1 value is " +s1+ " and n2 is " +s2);

pw.close();

  }

 }
```

web.xml

```
<web-app>

<context-param>

<param-name> number1 </param-name>
```

```
<param-value> 100 </param-value>
</context-param>
<context-param>
    <param-name> number2 </param-name>
    <param-value> 200 </param-value>
</context-param>
<servlet>
    <servlet-name>onServletContext</servlet-name>
    <servlet-class>com.tcs.OnServletContext</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>onServletContext</servlet-name>
    <url-pattern>/onContext</url-pattern>
</servlet-mapping>
    <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    </welcome-file-list>
</web-app>
```

2.4.3 Servlet Config

servlet using the web.xml deployment descriptor. For understanding, this is similar to a constructor in a java class.

1. In order to develop servlets we make use of web.xml and read the parameters using ServletConfig.
2. ServletConfig object exist one per servlet program.
3. An object of ServletConfig created by the container during its initialization phase. An object of ServletConfig is available to the servlet during its execution, once the servlet execution is completed, automatically ServletConfig interface object will be removed by the container.
4. An object of ServletConfig interface contains <init-param> details at web.xml, of a particular servlet.
5. An object of ServletConfig is created by the web container for each servlet during initialization of the servlet that is during the init() method call
6. An object of ServletConfig contain the <init-param> data in the form of key, value pairs, here the keys represents init param names and values are its values, which are represented in the web.xml file

An object of ServletConfig can be obtained in two ways, they are:

1. Using the getServletConfig() method directly

```
ServletConfigconf = getServletConfig();
```

In the above statement, we are directly calling getServletConfig() method as it is available in Servlet interface, inherited into GenericServlet and defined and further inherited into HttpServlet and later inherited into our own servlet class.

2. ServletConfig object will be available in init() method of the servlet.

```
public void init(ServletConfig config)
```

```
{// ……                    }
```

So finally we are able to create ServletConfig object in our servlet class, then how to get the data from .

Example:

index.html

```
<form action="calServlet" method="post">
Example on ServletConfig<br>
<input type="submit" value="Calculate">
</form>
```

CalculateServlet.java

```
package com.tcs;
public class CalculateServlet extends HttpServlet
 {
{        protected void doPost(HttpServletRequestreq,HttpServletResponse
                                res)throws ServletException,IOException
  {
  {      PrintWriter pw=res.getWriter();
res.setContentType("text/html");
```

```
ServletConfigconf=getServletConfig();
        String s1=conf.getInitParameter("number1");
        String s2=conf.getInitParameter("number2");
pw.println("First number value is " +s1+ " and Second number value is is " +s2);
pw.close();
    }
  }
```

web.xml

```
<web-app>
<servlet>
<servlet-name>onServletConfig</servlet-name>
<servlet-class>com.tcs.CalculateServlet</servlet-class>
<init-param>
<param-name> number1 </param-name>
<param-value> 100 </param-value>
</init-param>
<init-param>
<param-name> number2 </param-name>
<param-value> 200 </param-value>
```

```
</init-param>
</servlet>
<servlet-mapping>
<servlet-name>onServletConfig</servlet-name>
<url-pattern>/onSCG</url-pattern>
</servlet-mapping>
<welcome-file-list>
<welcome-file>index.html</welcome-file>
</welcome-file-list>
</web-app>
```

2.4.4 ServletRequest Object and its Methods

ServletRequest Object is blueprint of an object to provide client request information to a servlet. The servlet container creates a ServletRequest object and sends it as an argument to the servlet's service method. An object of ServletRequest is used to provide the client request information to a servlet such as content type, content length, parameter names and values, header information, attributes etc.

Methods and Definitions:

Important methods of ServletRequest:

Some important methods of ServletRequest	
Methods	**Description**
`Object getAttribute(String name)`	return attribute set on request object by name
`Enumeration getAttributeName()`	return an Enumeration containing the names of the attributes available inthis request
`int getContentLength()`	return size of request body
`int getContentType()`	return media type of request content
`ServletInputStream getInputStream()`	returns a input stream for reading binary data
`String getParameter(String name)`	returns value of parameter by name
`String getLocalAddr()`	returns the Internet Protocol(IP) address of the interface on which the request was received
`Enumeration getParameterNames()`	returns an enumeration of all parameter names

Figure 9: Methods of ServletRequest

`ServletContext getServletContext()`	return the servlet context of current request.
`String getServerName()`	returns the host name of the server to which the request was sent
`int getServerPort()`	returns the port number to which the request was sent
`boolean isSecure()`	returns a boolean indicating whether this request was made using a secure channel, such as HTTPS.
`void removeAttribute(String name)`	removes an attribute from this request
`void setAttribute(String name, Object o)`	stores an attribute in this request.

Figure 10: ServletRequest Methods

Important methods of HttpServletRequest

HttpServletRequest interface adds the methods that relates to the HTTP protocol.

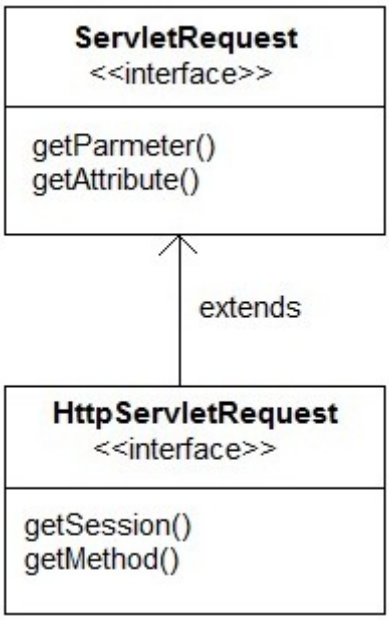


Figure 11: Hierarchy of HttpServletRequest

Some important methods of HttpServletRequest

Methods	Description
`String getContextPath()`	returns the portion of the request URI that indicates the context of the request
`Cookies getCookies()`	returns an array containing all of the Cookie objects the client sent with this request
`String getQueryString()`	returns the query string that is contained in the request URL after the path
`HttpSession getSession()`	returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session
`String getMethod()`	Returns the name of the HTTP method with which this request was made, for example, GET, POST, or PUT.

Figure 12: Methods of HttpServletRequest

Example of HttpServletRequest :

index.html:

```
<form action="welcome" method="get">

Enter your name<input type="text" name="name"><br>

<input type="submit" value="login"></form>
```

DemoServ.java

```
importjavax.servlet.*;

import java.io.*;

public class DemoServ extends HttpServlet{

public void doGet(HttpServletRequestreq,HttpServletResponse res)

throws ServletException,IOException

{

res.setContentType("text/html");

PrintWriter pw=res.getWriter();

String name=req.getParameter("name");//will return value

pw.println("Welcome "+name);

pw.close();

}}
```

2.4.5 ServletResponse Object and its Methods

It is the blueprint of an object to assist a servlet in sending a response the client. The servlet container creates a ServletResponse object and passes it as an argument to the servlet's

service method. Data that needs to be send to the client will be put inside the ServletResponse object. To send binary data back to the client in a MIME body response, use the ServletOutputStream from the ServletResponse object by calling the getOutputStream() method . To send character data to the client, the PrintWriter object returned by getWriter() should be used

.Important Methods of ServletResponse

Some important methods of ServletResponse

Methods	Description
`PrintWriter getWriter()`	returns a PrintWriter object that can send character text to the client.
`void setBufferSize(int size)`	Sets the preferred buffer size for the body of the response
`void setContentLength(int len)`	Sets the length of the content body in the response In HTTP servlets, this method sets the HTTP Content-Length header
`void setContentType(String type)`	sets the content type of the response being sent to the client before sending the respond.
`void setBufferSize(int size)`	sets the preferred buffer size for the body of the response.
`boolean isCommitted()`	returns a boolean indicating if the response has been committed

Figure 12: Methods of ServletResponse

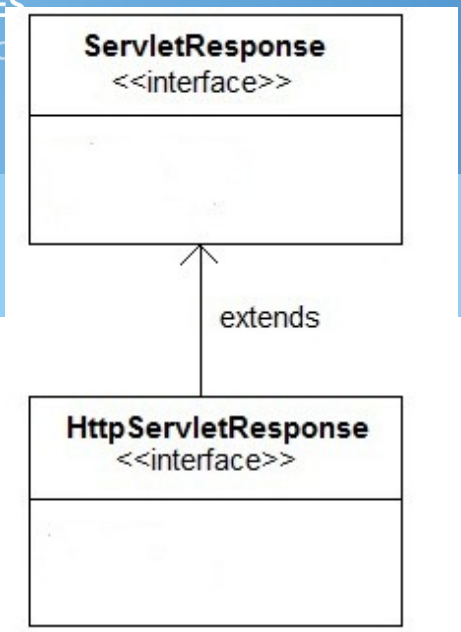


Figure 13: HttpServletResponse Hierarchy

Important methods of HttpServletResponse

HttpServletResponse interface adds the methods that relates to the HTTP response.

Some important methods of HttpServletResponse	
Methods	**Description**
`void addCookie(Cookie cookie)`	adds the specified cookie to the response.
`void sendRedirect(String location)`	Sends a temporary redirect response to the client using the specified redirect location URL and clears the buffer
`int getStatus()`	gets the current status code of this response
`String getHeader(String name)`	gets the value of the response header with the given name.
`void setHeader(String name, String value)`	sets a response header with the given name and value

Figure 14: HttpServletResponse Methods

Example of HttpServletResponse:

TheHttpResponse object has a lot of methods, so we cover the code snippet for most commonly used methods here. The rest can be can be read from the JavaDoc , if you are interested

The parts covered are:

- Writing HTML

- Content-Type
- Writing Text
- Content-Length
- Writing Binary Data
- Redirecting to a Different URL

Writing HTML

To send HTML back to the browser, you have to obtain the aPrintWriter from the HttpResponse object.

Here is how:

```
PrintWriter writer = response.getWriter();

writer.write("<html><body>GET/POST response</body></html>");
```

Headers

Just like the request object, the HttpResponse can contain HTTP headers. Headers must be set before any data is written to the response. You set a header on the response object like this:

```
response.setHeader("Header-Name", "Header Value");
```

As you can see, a response header is a name, value pair.

Content-Type

The Content-Type header is a response header that tells the browser the type of the content you are sending back to it. For instance, the content type for HTML is text/html. Similarly, if what you send back to the browser is plain text, you use the content type text/plain.

Here is how you set the Content-Type header on the HttpResponse object:

```
response.setHeader("Content-Type", "text/html");
```

Writing Text

You can write text back to the browser instead of HTML, like this:

```
response.setHeader("Content-Type", "text/plain");

PrintWriter writer = response.getWriter();

writer.write("This is just plain text");
```

```
PrintWriter writer = response.getWriter();

writer.write("This is just plain text");
```

First the Content-Type header is set to text/plain. Then a plain text string is written to the writer obtained from the response object.

Content-Length

The Content-Length header tells the browser how many bytes your servlet is sending back. If you are sending binary data back you need to set the content length header. Here is how:

```
response.setHeader("Content-Length", "31642");
```

Writing Binary Data

You can also write binary data back to the browser instead of text. For instance, you can send an image back, a PDF file or a Flash file or something like that.Again, you will first have to set the Content-Type header to the type matching the data you are sending back. For instance, the content type for a PNG image is image/png.

You can search for "mime types" in your favorite search engine to find a list of mime types (content types), so you can find the mime type for the content you are sending back.

In order to write binary data back to the browser you cannot use the Writer obtained from response.getWriter(). After all, Writer's are intended for text.

Instead you have to use the OutputStream obtained from the response.getOutputStream() method.

Here is how:

```
OutputStreamoutputStream = response.getOutputStream();

outputStream.write(...);
```

2.4.6 Servlet and Filter configuration in web.xml

2.4.6.1 Configuring Servlets

The web.xml defines mappings between URL paths and the servlets that handle requests with those paths. The web server uses this configuration to identify the servlet to handle a given request and call the class method that corresponds to the request method (e.g. the doGet() method for HTTP GET requests).

To map a URL to a servlet, you declare the servlet with the **<servlet>** element, then define a mapping from a URL path to a servlet declaration with the **<servlet-mapping>** element.

The **<servlet>** element declares the servlet, including a name used to refer to the servlet by other elements in the file, the class to use for the servlet, and initialization parameters. You can declare multiple servlets using the same class with different initialization parameters. The name for each servlet must be unique across the deployment descriptor. An example of a servlet configured in the web.xml is given below:

```
<servlet>
    <servlet-name>redteam</servlet-name>
    <servlet-class>mysite.server.TeamServlet</servlet-class>
    <init-param>
      <param-name>teamColor</param-name>
      <param-value>red</param-value>
    </init-param>
    <init-param>
      <param-name>bgColor</param-name>
      <param-value>#CC0000</param-value>
    </init-param>
  </servlet>

  <servlet>
    <servlet-name>blueteam</servlet-name>
    <servlet-class>mysite.server.TeamServlet</servlet-class>
    <init-param>
      <param-name>teamColor</param-name>
      <param-value>blue</param-value>
    </init-param>
```

```
    <init-param>
        <param-name>bgColor</param-name>
        <param-value>#0000CC</param-value>
    </init-param>
</servlet>
```

Note: More on Servlets would be dealt in the 'Introduction to Servlets' chapter.

The **<servlet-mapping>** element specifies a URL pattern and the name of a declared servlet to use for requests whose URL matches the pattern. The URL pattern can use an asterisk (*) at the beginning or end of the pattern to indicate zero or more of any character. (The standard does not support wildcards in the middle of a string, and does not allow multiple wildcards in one pattern.) The pattern matches the full path of the URL, starting with and including the forward slash (/) following the domain name. An example of servlet mapping with the servlet configured in the earlier step is given below:

```
<servlet-mapping>
    <servlet-name>redteam</servlet-name>
    <url-pattern>/red/*</url-pattern>
  </servlet-mapping>

  <servlet-mapping>
    <servlet-name>blueteam</servlet-name>
    <url-pattern>/blue/*</url-pattern>
  </servlet-mapping>
```

With this example, a request for the URL http://www.example.com/blue/teamProfile is handled by the TeamServlet class, with the teamColor parameter equal to blue and the bgColor parameter equal to #0000CC. The servlet can get the portion of the URL path matched by the wildcard using the ServletRequest object's getPathInfo() method.

Note: Static files, files that are served verbatim to users such as images, CSS or JavaScript, are handled separately from paths mentioned in the deployment descriptor. A request for a URL path that matches a path to a file in the WAR that's considered a static file will serve the file, regardless of servlet and filter mappings in the deployment descriptor.

The servlet can access its initialization parameters by getting its servlet configuration using its own getServletConfig() method, then calling the getInitParameter() method on the configuration object using the name of the parameter as an argument.

```
String teamColor = getServletConfig().getInitParameter("teamColor");
```

2.4.6.2 Configuring Filters

Similar to servlets, you configure a filter in the deployment descriptor by declaring the filter with the **<filter>** element, then mapping it to a URL pattern with the **<filter-mapping>** element. You can also map filters directly to other servlets.

The **<filter>**element contains a **<filter-name>**, **<filter-class>**, and optional **<init-param>** elements.

```
<filter>
   <filter-name>logSpecial</filter-name>
   <filter-class>mysite.server.LogFilterImpl</filter-class>
   <init-param>
     <param-name>logType</param-name>
     <param-value>special</param-value>
   </init-param>
 </filter>
```

The<filter-mapping> element contains a <filter-name> that matches the name of a declared filter, and either a <url-pattern> element for applying the filter to URLs, or a <servlet-name> element that matches the name of a declared servlet for applying the filter whenever the servlet is called.

```
<!-- Log for all URLs ending in ".special" -->
   <filter-mapping>
      <filter-name>logSpecial</filter-name>
      <url-pattern>*.special</url-pattern>
   </filter-mapping>
   <!-- Log for all URLs that use the "comingsoon" servlet -->
   <filter-mapping>
      <filter-name>logSpecial</filter-name>
      <servlet-name>comingsoon</servlet-name>
   </filter-mapping>
```

Note: Filters are not invoked on static assets, even if the path matches a filter-mapping pattern. Static files are served directly to the browser.

2.4.7 Initialization and Context Parameters in web.xml

2.4.7.1 Initialization Parameters

You define initialization attributes for servlets in the Web application deployment descriptor, web.xml, in the init-param element of the servlet element, using param-name and param-valuetags. If any value needs to be constant throughout the life of a servlet, it can be initialized in the web.xml file.

Init-param is defined inside a servlet element. This means it is only available to the servlet under declaration and not to other parts of the web application. If you want this parameter to be available to other parts of the application say a JSP this needs to be explicitly passed to the JSP. For instance passed as request.setAttribute(). This is highly inefficient and difficult to code. So if you want to get access to global values from anywhere within the application without explicitly passing those values, you need to use Context Init parameters.

```
<servlet>
```

```
  <servlet-name>controlServlet</servlet-name>
  <servlet-class>com.jenkov.butterfly.ControlServlet</servlet-class>
    <init-param>
      <param-name>myParam</param-name>
      <param-value>paramValue</param-value>
  </init-param>
</servlet>
```

Here is how you read the init parameters from inside your servlet - in the servlets init() method:

```
public class SimpleServlet extends GenericServlet {
 protected String myParam = null;
 public void init(ServletConfig servletConfig) throws ServletException{
  this.myParam = servletConfig.getInitParameter("myParam");
 }
 public void service(ServletRequest request, ServletResponse response)
      throws ServletException, IOException {
  response.getWriter().write("<html><body>myParam = " +
          this.myParam + "</body></html>");
 }
}
```

2.4.7.2 Context Parameters

This context param is available to all parts of the web application and it can be retrieved from the Context object. The context parameter would be available globally for all servlets to access.

```
<context-param>
    <param-name>myParam</param-name>
    <param-value>the value</param-value>
</context-param>
```

Here is how you access the parameter from inside an HttpServlet subclass:

```
String myContextParam = request.getSession().getServletContext().getInitParameter("myParam");
```

2.4.8 Project Directory Structure

A web application can be developed having the folder structure as mentioned.

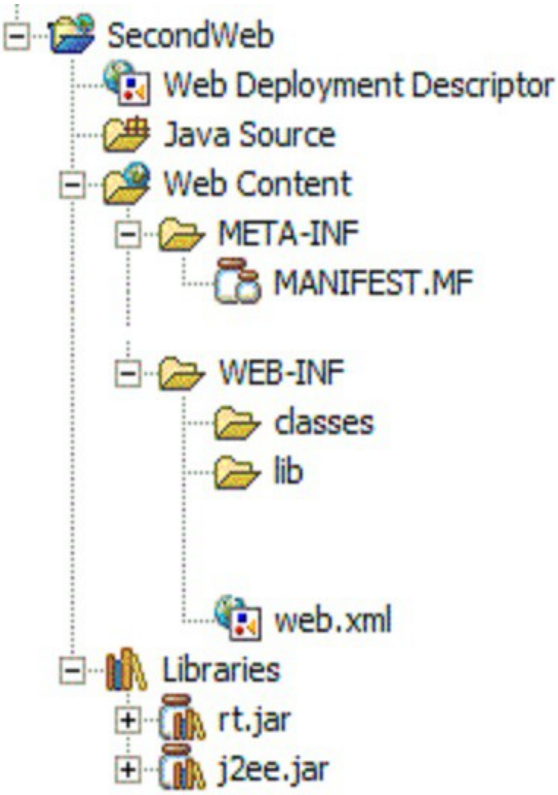


Figure 1: Project Directory Structure

Web Deployment Descriptor - The standard Web application deployment descriptor (web.xml)

Java Source – This Contains the project's Java source code for classes, beans, and servlets. When these resources are added to a Web project, they are automatically compiled and the generated files are added to the WEB-INF/classes directory.

Web Content - The mandatory location of all Web resources, including HTML, JSP, graphic files, and so on. If the files are not placed in this directory (or in a subdirectory structure under this directory), the files will not be run on the server. The Web Content folder represents the contents of the WAR file that will be deployed to the server. Any files not under the Web Content folder are considered development-time resources (for example, .java files, .sql files, and .mif files), and are not deployed when the project is unit tested or published.

META-INF - This directory contains the MANIFEST.MF file, which is used to map classpaths for dependent JAR files that exist in other projects in the same Enterprise Application project. An entry in this file will update the run-time project classpath and Java build settings to include the referenced JAR files.

WEB-INF - Based on the Sun Microsystems Java Servlet 2.3 Specification, this directory contains the supporting Web resources for a Web application, including the web.xml file and the classes and lib directories.

/classes - This directory is for servlets, utility classes, and the Java compiler output directory. The classes in this directory are used by the application class loader to load the classes. The .class files are placed in this directory automatically when the java compiler compiles java source files that are in the Java Source directory.

/lib - The supporting JAR files that your Web application references. Any classes in .jar files placed in this directory will be available for your Web application

Libraries - The supporting JAR files that your Web application references. This folder mirrors the content of the lib folder. In addition, Web Library Projects, which are "virtual" JAR files that do not physically reside in the Web project, but are associated with Java projects elsewhere in your workspace, are included in this folder. They are packaged with your project when you export the application's WAR file.

Note: For more details on Java Web Project,servlet and web.xml, refer the support document 2_Content - Introduction to Servlet_JavaWebProjectAndWebxml_SupportDoc.pdf

2.4.9 Summary

In this topic, you learnt about the below

- ✓ Types of Java Web Projects
- ✓ The Deployment Descriptor – web.xml and its elements
- ✓ Configuration of servlets and filters in the web.xml
- ✓ Context and Initialization Parameters in a servlet
- ✓ Project Directory Structure